

SVM & Logistic Regression for Wine-Quality Prediction

Davide Bertalero

1 Introduction

The purpose of this project is to investigate the use of two binary classification algorithms for the prediction of wine quality using data from physicochemical tests. Both the classic and the kernel versions of Support Vector Machine (SVM) and Logistic Regression (LR) algorithms have been implemented from scratch and used to predict wine quality through binary classification. The project has been implemented as a single jupyter notebook, containing code, explanations, visualizations and results in a similar fashion to this report, with additional plots not included here for brevity.

In section 2, the dataset is briefly examined and data preprocessing is described. Then, in section 3, the classical, linear versions of SVM and LR are presented, and tuning results are analyzed. In section 4, the kernel-based variants of the algorithms are discussed and evaluated, comparing the results with the classic, linear versions. Finally, section 5 concludes with final remarks.

The results show that both SVM and LR achieve similar performance on the dataset of ≈ 76 - 77% accuracy, with SVM slightly outperforming LR, with some minor improvement ($\approx 1\%$) for the kernelized SVM and no improvement for the kernelized LR, possibly due to insufficient tuning of the latter. The kernelized versions of the algorithms show potential for improved performance, but eventually achieve similar results to the classic versions, suggesting that the complexity of the underlying distribution could not be overcome by enabling non-linear modeling, or that additional tuning is required to fully exploit the kernel methods.

2 Data

2.1 Dataset

The dataset, available at <https://archive.ics.uci.edu/dataset/186/wine+quality> contains 6497 samples of two broad categories of wine, red wine and white wine. According to the related paper by Cortez et al. [1], the features represent the results of laboratory analysis, whereas the quality that we wish to predict is obtained as the median of subjective sensory evaluations.

We note that, after binarization (quality ≥ 6 vs. < 6), the dataset exhibits class imbalance, with about 63% positive labels and 37% negative labels, meaning there are ≈ 1.7 times more positive samples than negative samples. A naive classifier always predicting the positive class would therefore achieve $\approx 63\%$ accuracy, which is the baseline accuracy we consider to evaluate our models.

The data has been split into training and test sets, using an 80-20 split, meaning about 80% of the data is used for training and about 20% for testing. We use stratified splits so that the label distribution is maintained in both training and test subsets.

Looking at unscaled, original feature data shown in table 1, all 11 chemical features (thus excluding quality) are represented by positive and continuous scalar values.

Table 1: Dataset Feature Statistics

Feature	Min	Max	Mean	Std
fixed acidity	3.80	15.90	7.22	1.30
volatile acidity	0.08	1.58	0.34	0.16
citric acid	0.00	1.66	0.32	0.15
residual sugar	0.60	65.80	5.44	4.76
chlorides	0.01	0.61	0.06	0.04
free sulfur dioxide	1.00	289.00	30.53	17.75
total sulfur dioxide	6.00	440.00	115.74	56.52
density	0.99	1.04	0.99	0.00
pH	2.72	4.01	3.22	0.16
sulphates	0.22	2.00	0.53	0.15
alcohol	8.00	14.90	10.49	1.19
quality	3.00	9.00	5.82	0.87

2.2 Data preprocessing

To work on binary classification, integer quality scores (in this dataset ranging from 3 to 9) are preprocessed into two binary categories, encoded as -1 and $+1$ by convention. A sample is considered of a good wine when quality ≥ 6 , and of a bad wine when quality < 6 , i.e. the threshold is 6. Good wine is associated with the positive label $+1$, and bad wine with the negative label -1 .

Figure 1 shows the feature distributions by label, highlighting that most features exhibit high overlap between the two classes, with the exception of alcohol and density, which show more distinct distributions between the two classes. In particular, higher alcohol and lower density appear more likely related to good wine.

As shown in table 1, different features are distributed across different orders of magnitude and not centered on 0. We thus standardize them (i.e. center to 0 and scale to unit variance) to help optimization and training stability. We use the `StandardScaler` from `sklearn.preprocessing` to standardize features.

We fit the scaler only on the training data, and then transform both training data and test data. This way no test data information leaks into the scaling process, and all data is transformed in the same way, with distribution information coming only from the training data.

We also add a constant feature of fixed value 1, using the often called "bias trick". This is transparent to the algorithms, though the L2 regularization term in the optimization problems now includes the bias, i.e., $\|\mathbf{w}_{\text{bias trick}}\|^2 = \|\mathbf{w}_{\text{original}}\|^2 + b^2$, where b is the bias and $\mathbf{w}$ the learned vector of weights.

In the following, let $m = 5197$ denote the number of examples in the training set and let $d = 11 + 1 = 12$ denote the number of features, considering the extra bias feature as a standard feature.

Randomness is controlled with a fixed random seed for reproducibility, that can be set to reproduce results or left unset to have a random run using `numpy`'s random number generation.

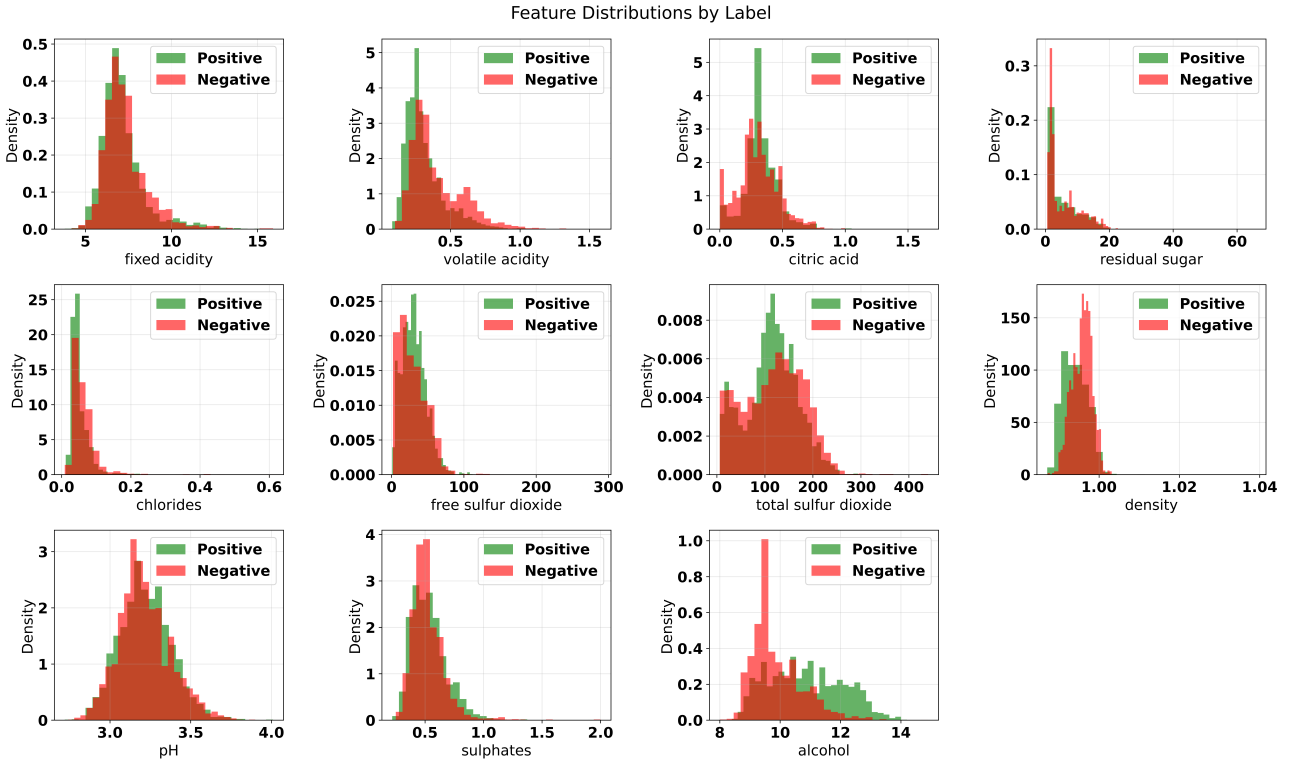


Figure 1: Feature distributions by label

3 Classic SVM and LR

3.1 Support Vector Machine

Objective

We implemented the Support Vector Machine (SVM) Pegasos algorithm, not assuming data to be linearly separable. Both the classic and the kernel versions in section 4 follow the original Pegasos paper by Shalev-Shwartz et al. [2].

The optimization problem we wish to solve is the following:

$$\min_{\mathbf{w} \in \mathbb{R}^d} F(\mathbf{w}) = \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{t=1}^m h_t(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2 = \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{t=1}^m l_t(\mathbf{w})$$

where

- $\mathbf{w} \in \mathbb{R}^d$ is a vector of weights to learn.
- $\frac{1}{m} \sum_{t=1}^m$ means it sums over each example t and averages $h_t(w)$.
- $h_t(\mathbf{w})$ is the hinge loss for example t , that is $h_t(\mathbf{w}) = [1 - y_t \mathbf{w}^\top \mathbf{x}_t]_+$
- $\lambda \in \mathbb{R} : \lambda > 0$ is a regularization parameter.
- $l_t(\mathbf{w}) = h_t(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$ is the strongly convex L2-regularized hinge loss function for example t .

The SVM algorithm computes a vector of weights $\mathbf{w}$ of size d .

SGD update and implementation

The implementation uses mini-batch Stochastic Gradient Descent (SGD) to minimize the objective function. The algorithm picks a set of random examples $A_t \subset \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ at each iteration t , estimates the subgradient on those examples, and updates the weight vector $\mathbf{w}$ in the opposite direction with a decreasing step size. The number of examples extracted at each step depends on the parameter $b = |A_t|$, the mini-batch size, which can be any $b \in \mathbb{N} : 1 \leq b \leq m$.

Vectorial operations are used to speed up computations, exploiting efficient `numpy` array operations and allowing quicker mini-batch computation.

At each iteration $\mathbf{w}$ is updated based on the mini-batch approximated subgradient of the objective function as follows:

$$\begin{aligned} \mathbf{w}_{t+1} &\leftarrow \mathbf{w}_t - \eta_t \nabla l_t^{A_t}(\mathbf{w}_t) \\ &= \mathbf{w}_t - \eta_t \left(\lambda \mathbf{w}_t - \frac{1}{b} \sum_{i \in A_t} \mathbf{1}[h_i(\mathbf{w}_t) > 0] y_i \mathbf{x}_i \right) \\ &= \mathbf{w}_t - \eta_t \left(\lambda \mathbf{w}_t - \frac{1}{b} \sum_{i \in A_t} \mathbf{1}[y_i \mathbf{w}_t^\top \mathbf{x}_i < 1] y_i \mathbf{x}_i \right) \end{aligned}$$

where $\nabla l_t^{A_t} = \lambda \mathbf{w}_t - \frac{1}{b} \sum_{i \in A_t} \mathbf{1}[y_i \mathbf{w}_t^\top \mathbf{x}_i < 1] y_i \mathbf{x}_i$ is the mini-batch approximation of the subgradient of the objective function, in which subgradients of the examples in the mini-batch are averaged on the mini-batch size b .

The step size for each update $\eta_t = \frac{1}{\lambda t}$ depends on the regularization parameter λ and the iteration number t . This schedule shrinks the step size to zero as the number of iterations grows, allowing the algorithm to converge as updates become progressively smaller.

Pegasos thus converges, in expectation, to the unique minimizer, up to stochastic error, because the objective is λ strongly convex. But SGD is inherently noisy and random, meaning

the result can vary from run to run. When tuning hyperparameters, training and testing are repeated for multiple runs to control the variance and get more reliable performance estimates.

During training, loss and accuracy histories are computed and saved, on the whole training set, for later visualization and inspection. A small helper function is used to control the sampling rate, to avoid computing loss and accuracy at each iteration for performance reasons, and was set to sample every 10 iterations.

Note a projection step, to project the weight vector onto a limited radius ball, is possible but not included in the implementation since it is not necessary in practice, as suggested in the original Pegasos paper [2].

Prediction

The prediction for a point $\mathbf{x}$ is computed taking the sign of the decision function (choosing +1 when $g(\mathbf{x}) = 0$):

$$\hat{y} = \text{sign}(g(\mathbf{x})) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

3.2 Logistic Regression

Objective

We implemented Logistic Regression (LR) as a Pegasos-style algorithm, accounting for the main difference from SVM: LR predicts the probability of the positive label, while SVM predicts sharply a positive or a negative label.

The optimization problem we wish to solve is similar to the SVM one, but with a different loss function. It is the following:

$$\min_{\mathbf{w} \in \mathbb{R}^d} F(\mathbf{w}) = \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{t=1}^m l'_t(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2 = \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{t=1}^m l_t(\mathbf{w})$$

where

- $l'_t(\mathbf{w}) = \ln(1 + e^{-y_t \mathbf{w}^\top \mathbf{x}_t})$ is the logistic loss for example t .
- $l_t(\mathbf{w}) = l'_t(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$ is the strongly convex L2-regularized logistic loss function for example t .

SGD update and implementation

We used mini-batch SGD like for SVM, adapting the gradient computation to the logistic loss. The update rule with the regularized logistic loss function becomes the following:

$$\begin{aligned} \mathbf{w}_{t+1} &\leftarrow \mathbf{w}_t - \eta_t \nabla l_t^{A_t}(\mathbf{w}_t) \\ &= \mathbf{w}_t - \eta_t \left(\lambda \mathbf{w}_t - \frac{1}{b} \sum_{i \in A_t} \frac{y_i \mathbf{x}_i}{1 + e^{y_i \mathbf{w}_t^\top \mathbf{x}_i}} \right) \\ &= \mathbf{w}_t - \eta_t \left(\lambda \mathbf{w}_t - \frac{1}{b} \sum_{i \in A_t} y_i \mathbf{x}_i \sigma(-y_i \mathbf{w}_t^\top \mathbf{x}_i) \right) \end{aligned}$$

where $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is the logistic (or sigmoid) function. Note that our implementation uses two equivalent definitions of $\sigma(\cdot)$ for numeric reasons. The first version is numerically stable w.r.t. highly negative z , whereas the second w.r.t. highly positive z . A stable computation chooses the best one to avoid overflow based on the value of z :

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z} \in (0, 1)$$

Prediction

LR allows to predict $\eta(x) = \mathbb{P}(Y = 1|X = x)$ the probability of a positive label, and of a negative label as $1 - \mathbb{P}(Y = 1|X = x)$.

We can apply a threshold to the probability to recover a binary prediction from the decision score, using 0 as threshold, equivalent to estimated probability ≥ 0.5 of the positive class.

We can thus both predict the probability $\eta(x)$ as $\hat{\eta}(x) = \sigma(g(\mathbf{x})) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}}}$, or make a binary prediction, as with SVM, taking the sign of the decision function $g(\cdot)$:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

3.3 Testing

Framework

We tested both SVM and LR using 5-fold Cross Validation for parameter tuning. The hyperparameters to tune were the same for both algorithms: the regularization coefficient λ and the batch size b . We chose to automatically compute the number of iterations T as $T = \frac{1}{\epsilon\lambda} \log(\frac{1}{\epsilon\lambda})$ so that T grows with $\frac{1}{\lambda}$ and $\frac{1}{\epsilon}$ and it is $\tilde{O}(\frac{1}{\epsilon\lambda})$ ¹ as suggested in the original Pegasos paper [2], where ϵ is the desired accuracy of the solution, fixed to $\epsilon = 0.1$ in our experiments.

Due to performance (not too many iterations) and convergence (not too few iterations) reasons, T values are clipped so that $T \geq 1000$ and $T \leq 20000$. With these limits it becomes feasible to run the tuning with all such T values for each (λ, b) pair.

Remarkably, neither T nor the training runtime depend on the dataset size m .

We use 5-fold CV with 3 random repetitions for each hyperparameter combination to find the best hyperparameters while controlling variance. We select the best hyperparameters using the average accuracy over folds and repetitions. A simple average is used to choose the best parameters, but other methods, possibly taking into account the variance, exist, and may be explored to extend this work.

Then the model is trained on the whole training set, using the best hyperparameters found, saving the loss and accuracy history for visualization. Note a newly generated random seed is used for the final training to have a new, random, run, not reusing the best seed found in tuning to avoid overfitting to the tuning process.

We tested the following hyperparameter values, exponentially spaced to cover a wide range of values while keeping the number of combinations manageable:

- $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10^1, 10^2, 10^3\}$
- $b \in \{1, 10, 100\}$
- T computed as above, meaning in practice $T \in \{1000, 6908, 20000\}$

These amount to 63 hyperparameter combinations to test for each algorithm, with a total of 189 training runs. The total tuning runtime was about ≈ 2.5 minutes for SVM and ≈ 4 minutes for LR on the same machine.

Best SVM and LR results

Figures 2-5 show the performance of the models trained with the tuned hyperparameters on training and test data, while figure 6 shows a comparison of the metrics for the best SVM and LR models on test data. The considered metrics are accuracy, precision, recall and F1-score.

The best performing hyperparameters found with cross validation are the following:

¹ $\tilde{O}(\cdot)$ ignores logarithmic factors

- SVM: $\lambda = 0.001$, $b = 100$, $T = 20000$
- LR: $\lambda = 0.001$, $b = 100$, $T = 6908$

The best SVM and LR models achieve similar performance, respectively 76.15% and 75.69% accuracy on test data, with SVM slightly outperforming LR on all the considered metrics, as can be seen in figure 6.

In particular, the more detailed notebook output shows that different hyperparameter combinations achieve similar performance in the tuning process, with 12/63 combinations for SVM and 13/63 for LR within 1% accuracy of the best mean accuracy found. This suggests that the models are not too sensitive to hyperparameter choice within a reasonable range, and techniques to choose hyperparameters taking more factors into account (e.g., variance) could be explored to improve reliability without performance degradation.

The best performing hyperparameter combinations have small λ , that is $\lambda \in \{10^{-3}, 10^{-2}\}$, but different batch sizes $b \in \{1, 10, 100\}$ and iteration numbers $T \in \{1000, 6908, 20000\}$, showing that regularization plays a more significant role in model performance, with lower values allowing for greater accuracy.

The confusion matrices in figures 2-3 show similar performance (within 1% accuracy) between training and test data for both models. They also show that both models are better at predicting the positive class, which is expected given the dataset imbalance towards the positive class. Both models achieve similar training and test scores in the metrics comparison, with SVM slightly outperforming LR.

The loss and accuracy plots show reached convergence for both models, with small oscillations in the final iterations. This suggests no overfitting is present in the models, as also confirmed by the similar training and test scores.

Underfitting is limited, as both models show a $\approx +13\%$ improvement from the naive baseline. Still, the $\approx 76\%$ accuracy on test data suggests the models may not be complex enough to fully capture the underlying patterns in the data.

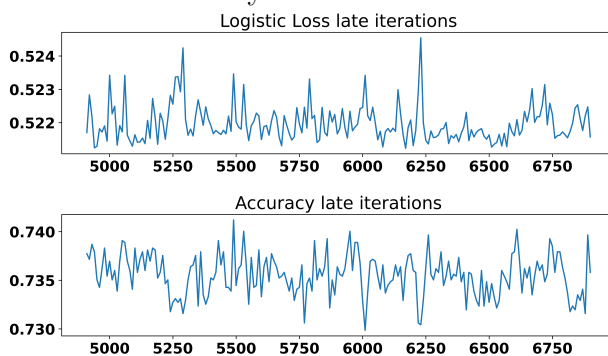
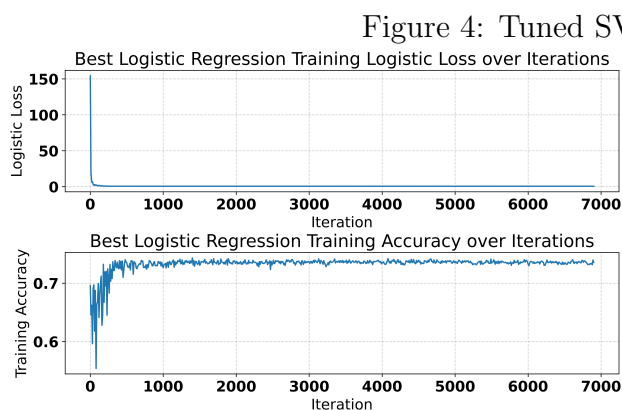
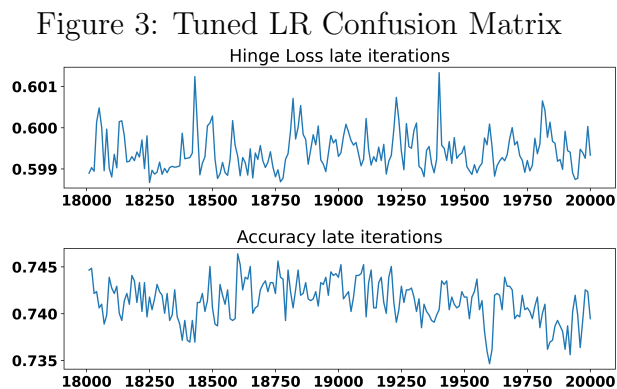
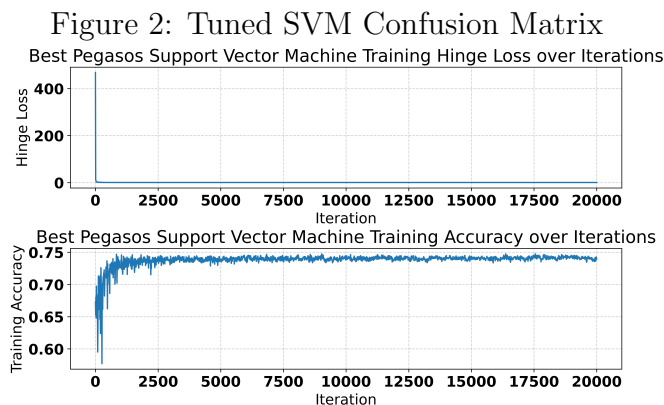
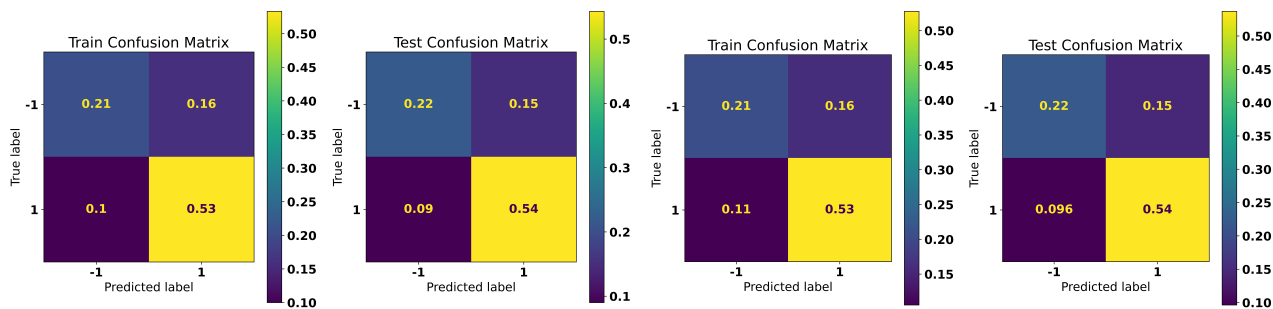


Figure 4: Tunned SVM Loss and Accuracy

Figure 5: Tunned LR Loss and Accuracy

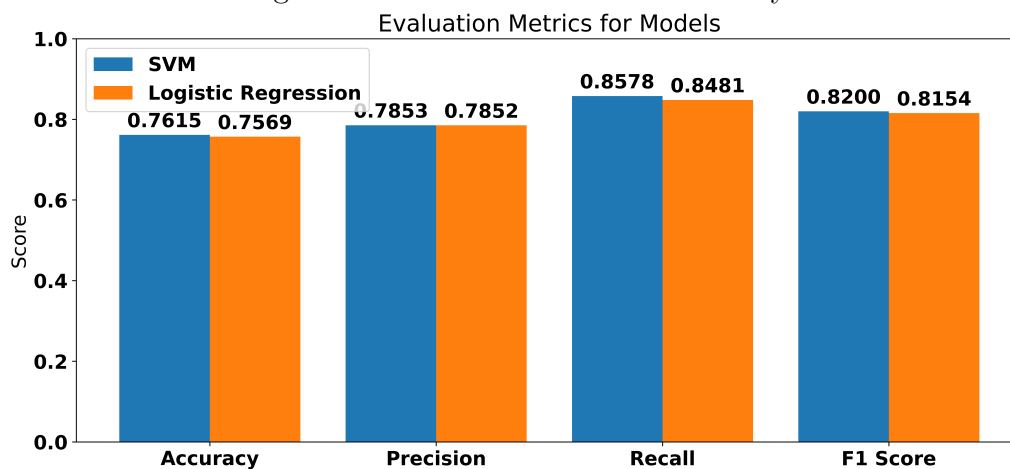


Figure 6: Metrics for tuned SVM and LR

4 Kernel SVM and LR

We implemented a kernelized version of the SVM and the LR algorithms, extending them to handle not linearly separable data via kernel methods. This could enable the models to learn more complex decision boundaries and achieve greater accuracy. In particular, kernel methods allow to implicitly map data points to a higher-dimensional space, where they may be more easily separable by a linear decision boundary, allowing the models to learn more complex, non-linear decision boundaries in the original space. Computationally, we don't need to explicitly compute the mapping $\phi(\cdot)$, but we can use kernel functions to compute dot products in the higher-dimensional space directly.

The representer theorem tells us that the optimal solution $\mathbf{w}$ to the optimization problem we're trying to solve can be expressed as a linear combination of the training instances with some coefficients α .

In particular, the weight vector $\mathbf{w}$ is stored implicitly as a linear combination of the mapped training examples:

$$\begin{aligned}\mathbf{w}_t &= \frac{1}{\lambda t} \sum_{i=1}^m \alpha_i y_i \phi(\mathbf{x}_i) = \frac{1}{\lambda t} \sum_{i=1}^m \beta_i \phi(\mathbf{x}_i) \\ \|\mathbf{w}\|^2 &= \sum_{s,r=1}^m \alpha_s \alpha_r y_s y_r K(\mathbf{x}_s, \mathbf{x}_r) = (\alpha \odot Y)^\top K(Y \odot \alpha) = \beta^\top K \beta\end{aligned}$$

where $\beta = \alpha \odot Y$ is the element-wise (Hadamard) product of α and Y , and K is the kernel Gram matrix with entry $K_{s,r} = K(\mathbf{x}_s, \mathbf{x}_r)$.

Note that, after training, the algorithm checks for zero coefficients α_i and prunes them, to save memory and speed up predictions. This is unlikely to produce significant speedup for our small dataset, but is easy to implement and has no negative effect on the training process or the final performance, so we included it in the implementation.

We can thus implicitly map points to a higher-dimensional space, using the feature map $\phi(\cdot)$ induced by a positive semidefinite (PSD) kernel K .

The kernel versions of the algorithms implemented don't use mini-batch SGD due to performance reasons, but could be extended to do so.

The kernel Gram matrix K with entry $K_{s,r} = K(\mathbf{x}_s, \mathbf{x}_r)$ with training example indices $1 \leq s, r \leq m$, is precomputed at the beginning of training to speed up training iterations at the cost of higher memory usage, storing a matrix of size $m \times m$. In fact, given the training set points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, the Gram matrix is constant, and can be precomputed once before training.

3 kernels were implemented, and are supported in both SVM and LR:

- the gaussian (Radial Basis Function) kernel $K_\gamma(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2 / (2\gamma))$: with parameter γ , the kernel width,
- the polynomial kernel $K_{d,c_0}(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c_0)^d$: with parameters d , the polynomial degree, and c_0 , a constant,
- the sigmoid kernel $K_{\gamma,c_0}(\mathbf{x}, \mathbf{x}') = \tanh(\gamma(\mathbf{x}^\top \mathbf{x}') + c_0)$: with parameters γ , the kernel scaling factor, and c_0 , a constant.

Note the sigmoid kernel is not guaranteed to be PSD for all values of γ and c_0 , but is sometimes used in practice. In particular, Lin [3] suggests choosing $\gamma > 0$ and $c_0 < 0$. We follow this suggestion in our implementation, but assume the kernel is not PSD anyway, and use a heuristic to avoid computing the RKHS norm of a possibly non PSD kernel Gram matrix.

The following discussion focuses on the two PSD kernels, that is the gaussian and the polynomial kernel, but the implementation supports all three kernels, with the special case for the sigmoid kernel.

The use of kernels introduces additional hyperparameters to tune, and increases the computational complexity of both training and prediction, due to the need to compute kernel evaluations and store the Gram matrix.

In particular, the iteration number T is still $\tilde{O}(\frac{1}{\epsilon\lambda})$ as in the classic case, but each iteration is more expensive, since it requires computing kernel evaluations with all training examples. The runtime is thus $\tilde{O}(\frac{m}{\epsilon\lambda})$, now dependent on the number of training examples m , differently from the classic versions of the algorithms.

Predicting is also more expensive, since it requires computing kernel evaluations with all support vectors, but a single prediction is still negligible for our dataset size.

Training kernel models is thus more expensive than training classic linear models. For instance, a single training run of classic SVM or LR with $T = 6908$ requires about 2s, while the kernel SVM with the same T requires about 60s on the same machine, a $\approx 30x$ increase in runtime.

Kernel LR is slower than kernel SVM, for two main reasons. First, each iteration updates the whole $\boldsymbol{\alpha}$ vector instead of the single α_i update used in kernel SVM. Secondly, the projection step is required in kernel LR to keep $\boldsymbol{\alpha}$ from exploding with the polynomial and the sigmoid kernels, which requires extra computations. This is not required by kernel SVM since update uses a different logic, which will be detailed in section 4.1. In particular, the projection step is $O(m^2)$ since it requires computing the RKHS norm of the $\boldsymbol{\alpha}$ vector, which involves a matrix-vector product with the kernel Gram matrix, and is required at each iteration. For instance, a single training run of kernel LR with the same $T = m = 6908$ requires about 80s for the gaussian or the polynomial kernel, a $\approx 1.3x$ increase in runtime w.r.t. kernel SVM and $\approx 40x$ increase in runtime w.r.t. classic SVM or LR. The sigmoid kernel LR is faster than other kernels, since the projection step is heuristic, saving computing time, requiring about 34s for $T = m = 6908$.

Kernel LR is thus $\tilde{O}(\frac{m^2}{\epsilon\lambda})$ for the gaussian and the polynomial kernel.

4.1 Kernel Support Vector Machine

Objective

In the kernel setting, the optimization problem for kernel SVM becomes:

$$\begin{aligned}\min_{\mathbf{w} \in \mathbb{R}^d} F(\mathbf{w}) &= \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{t=1}^m [1 - y_t \mathbf{w}^\top \phi(\mathbf{x}_t)]_+ + \frac{\lambda}{2} \|\mathbf{w}\|^2 \\ &= \min_{\boldsymbol{\alpha} \in \mathbb{R}^m} \frac{1}{m} \sum_{t=1}^m [1 - y_t (K\boldsymbol{\beta})_t]_+ + \frac{\lambda}{2} \boldsymbol{\beta}^\top K \boldsymbol{\beta}\end{aligned}$$

where:

- $K(\mathbf{x}_s, \mathbf{x}_t) = \phi(\mathbf{x}_s)^\top \phi(\mathbf{x}_t)$ is a PSD kernel function,
- α_i is the coefficient associated with training examples. Though $\boldsymbol{\alpha}$ coefficients are stored and updated, minimization still happens over $\mathbf{w}$ implicitly, via the representer theorem.

Given a current coefficient vector $\boldsymbol{\alpha}$, the decision function at iteration t for an input $\mathbf{x}$ is therefore computed as:

$$g_t(\mathbf{x}) = \mathbf{w}_t^\top \phi(\mathbf{x}) = \frac{1}{\lambda t} \sum_{i=1}^m \alpha_i y_i \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}) = \frac{1}{\lambda t} \sum_{i=1}^m \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) = \frac{1}{\lambda t} (\boldsymbol{\beta})^\top K_{\cdot, x}$$

SGD update and implementation

At each iteration t the algorithm samples a single training example $(\mathbf{x}_i, y_i)$ and computes its (scaled) margin $y_i g_t(\mathbf{x}_i)$. If the margin constraint is violated, that is if $y_i g_t(\mathbf{x}_i) < 1$, then the corresponding coefficient of that example is increased by 1:

$$[\alpha]_i \leftarrow [\alpha]_i + 1$$

Note α is initialized at $\mathbf{0}$.

In this kernel SVM implementation, the step size $\eta_t = \frac{1}{\lambda_t}$ is not used directly in the update, but is instead accounted for in the scaling factor of the decision function, as shown below. This is equivalent to using a decreasing step size in the updates.

Examples with $\alpha_i = 0$ do not contribute to the decision boundary, while those with $\alpha_i > 0$ act as support vectors.

Prediction

After training for T iterations, the learned decision function used to predict is:

$$g(\mathbf{x}) = \frac{1}{\lambda T} \sum_{i \in S} [\alpha]_i y_i K(\mathbf{x}_i, \mathbf{x})$$

where:

- S is the pruned set of indices with nonzero α_i
- $\frac{1}{\lambda T}$ is the scaling factor that substitutes the decreasing update rate in this implementation
- α coefficients, increased by 1 when updated, that count the number of times each example has been selected and had non-zero loss on it

and the predicted label is given by taking the sign: $\hat{y} = \text{sign}(g(\mathbf{x}))$.

4.2 Kernel Logistic Regression

Objective

The LR objective in the kernel setting becomes:

$$\begin{aligned} \min_{\alpha \in \mathbb{R}^m} F(\alpha) &= \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{t=1}^m \ln(1 + e^{-y_t \mathbf{w}^\top \phi(\mathbf{x}_t)}) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \\ &= \frac{1}{m} \sum_{t=1}^m \ln(1 + e^{-y_t \sum_{s=1}^m [\alpha]_s y_s K(\mathbf{x}_s, \mathbf{x}_t)}) + \frac{\lambda}{2} \beta^\top K \beta \end{aligned}$$

where the first term is the logistic loss and the second term is the L2 regularization on the coefficient vector α .

SGD update and implementation

The kernel Pegasos LR algorithm implemented performs SGD directly on the coefficient vector α , like in kernel SVM, but applies the step size $\eta_t = \frac{1}{\lambda_t}$ shrinkage directly to α . At each iteration t it samples a single training example $(\mathbf{x}_i, y_i)$, and updates α along the gradient of the regularized logistic loss, similarly to SGD used for classic SVM and LR, and unlike the count-based step size used in kernel SVM.

The update rule becomes:

$$\begin{aligned}\boldsymbol{\alpha}_{t+1} &\leftarrow \boldsymbol{\alpha}_t - \eta_t \nabla l_t(\boldsymbol{\alpha}_t) \\ &= \boldsymbol{\alpha}_t - \eta_t (\lambda Y \odot K(\boldsymbol{\alpha}_t \odot Y) - y_i \sigma(-y_i g_t(\mathbf{x}_i))(Y \odot K_{:,i}))\end{aligned}$$

Since updating $\boldsymbol{\alpha}$ directly resulted in exploding gradients in some cases, for the polynomial and the sigmoid kernels, a projection step was included. It keeps $\boldsymbol{\alpha}$ within a limited radius ball, similarly to the projection $\mathbf{w} \leftarrow \min\{1, \frac{1/\sqrt{\lambda}}{\|\mathbf{w}\|}\}\mathbf{w}$ suggested in the Pegasos paper [2] for classic SVM. In particular, we project $\boldsymbol{\alpha}$ onto the ball of radius $1/\sqrt{\lambda}$ in the induced RKHS norm by scaling $\boldsymbol{\alpha}$ as follows:

$$\boldsymbol{\alpha} \leftarrow \min\{1, \frac{1/\sqrt{\lambda}}{\sqrt{\|\mathbf{w}\|^2}}\}\boldsymbol{\alpha} = \min\{1, \frac{1/\sqrt{\lambda}}{\sqrt{\boldsymbol{\beta}^\top K \boldsymbol{\beta}}}\}\boldsymbol{\alpha}$$

Rather than constraining $\boldsymbol{\alpha}$ directly, we constrain the corresponding RKHS weight vector to satisfy $\|\mathbf{w}\| \leq \frac{1}{\sqrt{\lambda}}$. If the current $\boldsymbol{\alpha}$ violates this constraint after the update, that is if $\|\mathbf{w}\|^2 > \frac{1}{\lambda}$, we project back onto the boundary of the ball by rescaling as above. Since $\mathbf{w}$ is linear in $\boldsymbol{\alpha}$, this scaling ensures that the updated weight vector lies within the boundary of the admissible ball while preserving its direction.

Prediction

After training, only examples with non-zero coefficients α_i , the support vectors, contribute to the decision function. Given a new input $\mathbf{x}$, the kernel LR model computes the decision function, the probability estimate and binary prediction respectively as follows:

$$g(\mathbf{x}) = \sum_{i \in \mathcal{S}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}), \quad \hat{\eta}(\mathbf{x}) = \sigma(g(\mathbf{x})), \quad \hat{y} = \text{sign}(g(\mathbf{x}))$$

where $\mathcal{S}$ is the pruned set of indices of support vectors with non-zero α_i .

4.3 Testing

From initial test runs with fixed, arbitrary hyperparameters (i.e. $\lambda = 0.01$, $T = 6908$, $\gamma = 0.01$), we observed that the gaussian kernel SVM can easily overfit the training data, achieving $\geq 93\%$ training accuracy but with only $\approx 77\%$ test accuracy. Figures 7 and 8 illustrate this behavior, showing ongoing improvement in late iterations w.r.t. training set accuracy, without significant benefits in the test confusion matrix w.r.t. the classic SVM one in figure 2. This warns that careful hyperparameter tuning is necessary to achieve good generalization performance and avoid overfitting.

Regarding kernel choice, we expect the gaussian kernel to perform best, as it is a universal approximator and can model complex decision boundaries. The polynomial kernel may also perform well for certain degrees, while the sigmoid kernel's performance may vary depending on the chosen parameters as it isn't PSD.

As detailed above, the increased computational cost of kernel methods is evident in practice, with training runtime significantly longer than that of classic SVM and LR. Hyperparameters tuning was performed for both kernel SVM and LR, but was limited to fewer values for LR due to time constraints and the previously discussed computational complexity.

The following parameters were used in the hyperparameter tuning for kernel SVM:

- Common hyperparameters: $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10^1, 10^2, 10^3\}$, $T \in \{1000, m = 5197\}$
- gaussian Kernel: $\gamma \in \{0.01, 0.1, 1, 10, 100\}$, for a total of 70 combinations.

- Polynomial Kernel: $d \in \{1, 2, 4, 8\}$, $c_0 \in \{0.01, 0.1, 1, 10, 100\}$, for a total of 280 combinations.
- Sigmoid Kernel: $\gamma \in \{0.01, 0.1, 1, 10, 100\}$, $c_0 \in \{-100, -10, -1, -0.1, -0.01\}$, for a total of 350 combinations.

and the following for kernel LR:

- Common hyperparameters: $\lambda \in \{10^{-1}, 1, 10^1\}$, $T \in \{1000, 3500\}$
- Gaussian Kernel: $\gamma \in \{0.1, 1, 10\}$, for a total of 18 combinations.
- Polynomial Kernel: $d \in \{1, 2, 4\}$, $c_0 \in \{0.1, 1, 10\}$, for a total of 54 combinations.
- Sigmoid Kernel: $\gamma \in \{0.1, 1, 10\}$, $c_0 \in \{-10, -1, -0.1\}$, for a total of 54 combinations.

The total runtime was $\approx 9\text{m}$, $\approx 22\text{m}$, and $\approx 30\text{m}$ for the gaussian, polynomial and sigmoid kernel SVM respectively, and $\approx 36\text{m}$, $\approx 108\text{m}$, and $\approx 5.5\text{m}$ respectively for the gaussian, polynomial and sigmoid kernel LR.

The best hyperparameters, that is the ones achieving highest mean accuracy, found with cross validation are:

- SVM:
 - Gaussian Kernel SVM: $\lambda = 0.001, \gamma = 1, T = 5197$
 - Polynomial Kernel SVM: $\lambda = 1, d = 2, c_0 = 10, T = 5197$
 - Sigmoid Kernel SVM: $\lambda = 0.01, \gamma = 0.1, c_0 = -0.01, T = 5197$
- LR:
 - Gaussian Kernel LR: $\lambda = 1, \gamma = 0.1, T = 3500$
 - Polynomial Kernel LR: $\lambda = 0.1, d = 4, c_0 = 10, T = 1000$
 - Sigmoid Kernel LR: $\lambda = 1, \gamma = 10, c_0 = -1, T = 3500$

We can observe from figures 9 and 10 that the tuned gaussian kernel SVM achieves good performance on both training and test data, with limited overfitting, possibly due to the limited T values used in tuning, which may have limited model complexity. The confusion matrix in figure 9 shows similar performance to the classic SVM model in figure 2, while the loss and accuracy plots show convergence and limited overfitting.

Other kernel SVM and LR models show similar performance and behaviour, with worse results for the kernel LR models, likely due to the limited hyperparameter choices used in tuning, and are not discussed in detail here for brevity.

In particular, the polynomial kernel LR shows clear signs of underfitting, with accuracy of $\approx 37\%$, below the constant classifier baseline of $\approx 63\%$, likely due to the limited choice of hyperparameters used in tuning, and possibly due to the projection step being too constraining. In fact, the best polynomial kernel LR found with tuning was actually a constant negative classifier, showing it was not able to learn any useful pattern from the data, not even the bias towards positive labels. Observing the loss and accuracy plots in figure 11 for this model shows oscillations between two values that roughly correspond to the accuracy of the two different constant classifiers.

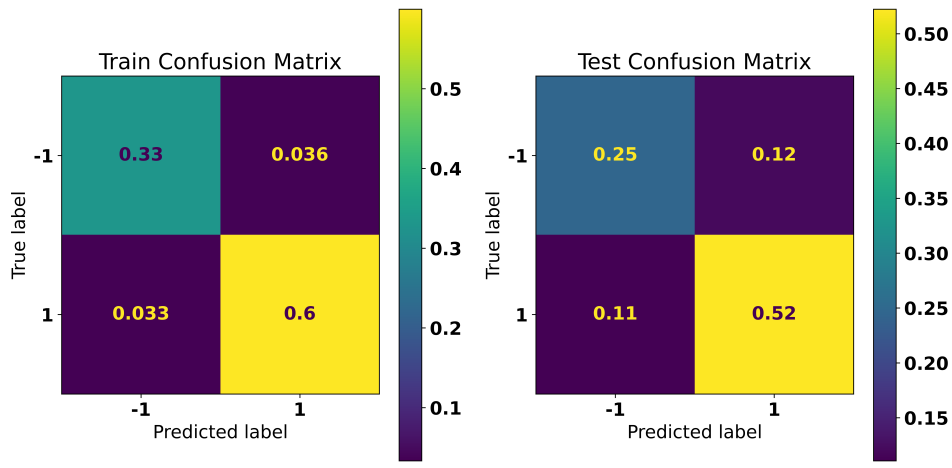


Figure 7: Gaussian kernel SVM Confusion Matrix, arbitrary hyperparameters

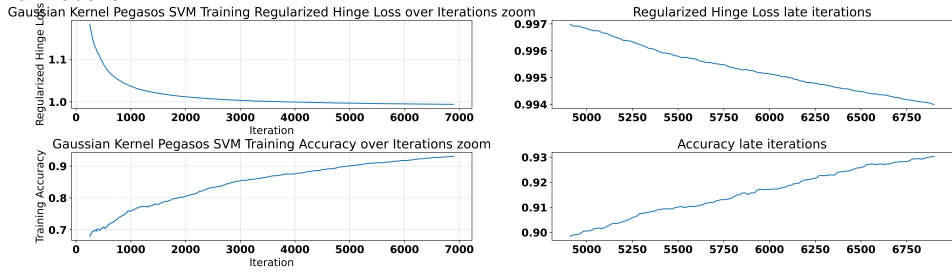


Figure 8: Gaussian kernel SVM Loss and Accuracy, arbitrary hyperparameters

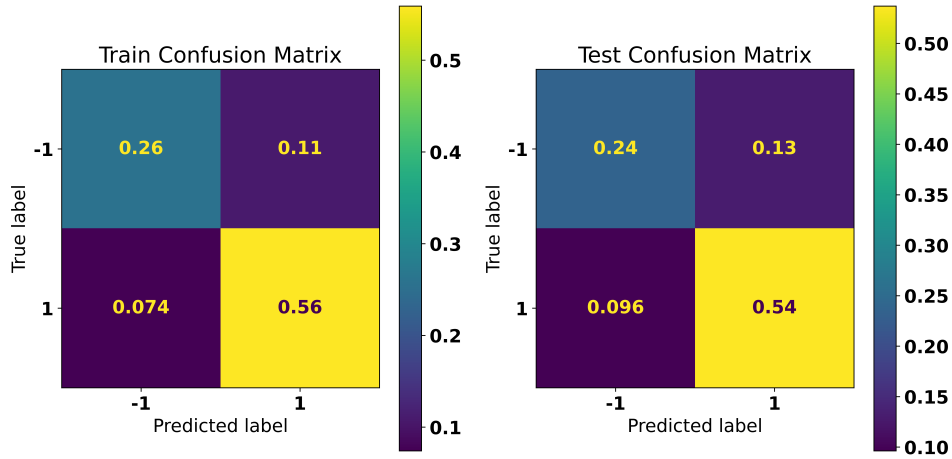


Figure 9: Tuned gaussian kernel SVM Confusion Matrix

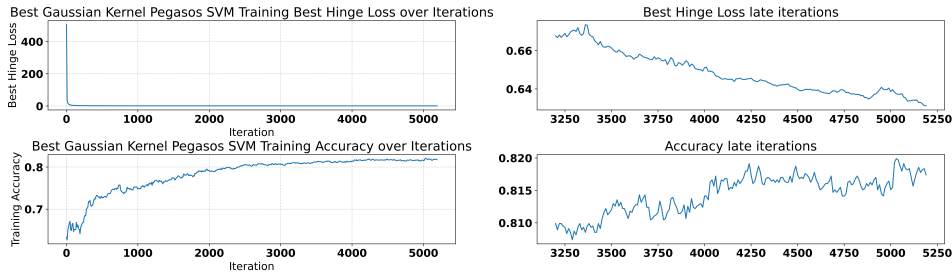


Figure 10: Tuned gaussian kernel SVM Loss and Accuracy

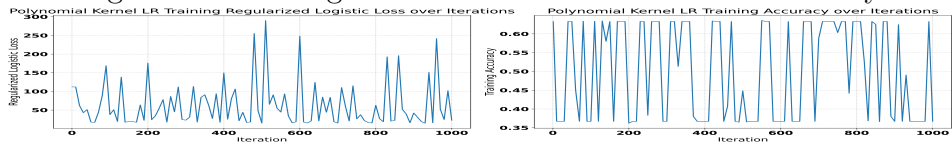


Figure 11: Tuned Polynomial kernel LR Loss and Accuracy underfit

Table 2 summarizes the performance of all tested models, including classic and kernel SVM and LR, ranked by accuracy on test data. The gaussian kernel SVM achieves the highest accuracy, followed by classic SVM and polynomial kernel SVM.

Sigmoid Kernel LR shows similar performance to classic LR, whereas gaussian and polynomial kernel LR perform worse, possibly due to the limited set of hyperparameter choices used in tuning and the low number of iterations.

The tuned performance of the kernel models is similar to the classic SVM and LR ones, with maximum accuracy of $\approx 76\text{-}77\%$ achieved on test data, suggesting that the increased model complexity from the kernels did not significantly improve performance on this dataset for the hyperparameter choices explored.

The confusion matrices in figure 12 show similar performance across all tuned models on test data, showing that for all models it is harder to predict the negative class, possibly due to class imbalance in the dataset. In particular, the gaussian kernel SVM slightly improves over classic SVM in predicting the negative class, from ≈ 0.22 to ≈ 0.24 true negatives, keeping about the same performance on the positive class of ≈ 0.54 true positives.

The final best model, the gaussian kernel SVM, was inspected in more detail to study its misclassifications. Feature distribution plots, over correct and misclassified, false positives and false negatives examples, omitted here for brevity, show overlap of the distributions. In particular, alcohol and density appear to be the most discriminative features, with misclassified examples having lower alcohol and higher density on average than correctly classified ones, but still with significant overlap between the distributions of the two groups. This aligns with the initial observations on the dataset in section 2.2, where most features showed significant overlap between the two classes, with mainly alcohol and density showing some separation between the two classes. This was confirmed by computing the mean and standard deviation of each distribution. In fact, the absolute mean difference between the two groups is ≈ 0.43 for alcohol and ≈ 0.36 for density, while other features have lower values below ≈ 0.16 , down to 0.018 for the least discriminative feature, free sulfur dioxide. This seems to confirm that there may be strong overlap between the two classes in the d -dimensional feature space, possibly explaining the limited performance of all models and the limited improvement achieved with kernels, which were not able to find significantly better decision boundaries.

Finally, inspecting the distribution of the decision score for the gaussian kernel SVM showed that, as expected, misclassifications are generally close to the decision boundary, with a mean distance of ≈ 0.50 .

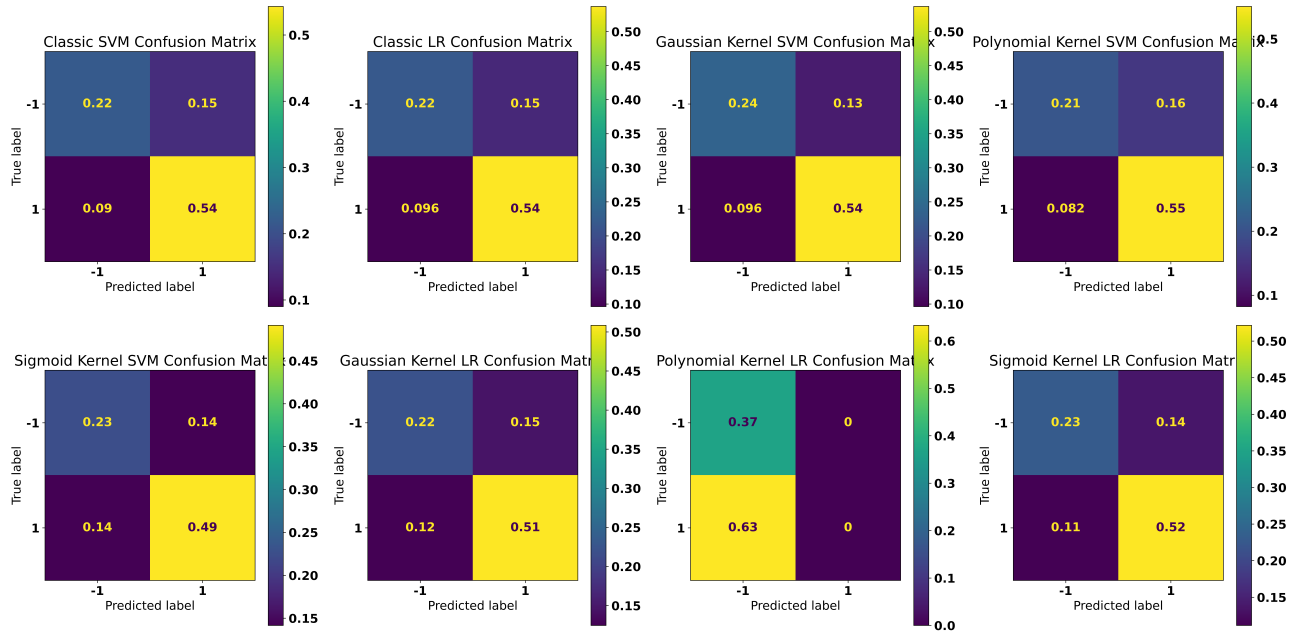


Figure 12: Confusion matrices of tuned models on test data.

Table 2: Model Performance on test data, ranked by accuracy

Model Name	Accuracy	Precision	Recall	F1 Score
Gaussian Kernel SVM	77.38	80.51	84.81	82.60
Classic SVM	76.15	78.53	85.78	82.00
Polynomial Kernel SVM	75.92	77.66	87.00	82.06
Classic LR	75.69	78.52	84.81	81.54
Sigmoid Kernel LR	75.15	79.21	82.38	80.76
Gaussian Kernel LR	73.00	77.76	80.32	79.02
Sigmoid Kernel SVM	71.69	77.64	77.64	77.64
Polynomial Kernel LR	36.69	0.00	0.00	0.00

5 Conclusions

In this work we have implemented and tested both the classic and the kernelized versions of SVM and LR algorithms using the Pegasos SGD approach for a binary classification task on a real-world dataset.

Regarding the classic, linear versions of the algorithms, we observed that both SVM and LR achieved similar performance, with SVM slightly outperforming LR, respectively achieving 76.15% and 75.69% accuracy on test data.

Overall, the experiments show that both classic and kernelized SVM and LR achieve similar performance, with the gaussian kernel SVM slightly outperforming the others, and some kernel algorithms performing worse than the classic ones, possibly due to the dataset characteristics or the hyperparameter choices made, the latter especially likely for LR.

In fact, the best model, the gaussian kernel SVM, improved on the classic SVM only marginally, from 76.15% to 77.38% accuracy on test data, while the best kernel LR, the sigmoid kernel LR, performed slightly worse than classic LR, thus still worse than SVM, from 75.69% to 75.15% accuracy on test data, likely due to the computing-limited hyperparameter choices used in tuning.

The final best model, the gaussian kernel SVM, was inspected in more detail, showing that misclassified examples are close to the decision boundary and that there is a strong overlap between the two classes in the feature space, possibly explaining the limited performance of all models and the limited improvement achieved with kernels.

Possible extensions of this work include:

- refining the iteration stopping criteria, possibly based on validation loss or accuracy, trying multiple final weight vectors at the last iterations, choosing the best one based on validation performance, or testing taking averages of the weight vectors.
- extending the hyperparameter tuning process (in particular properly tuning kernel LR with a wider range of values), exploring more granular hyperparameter values, or intensifying the search around promising values.
- exploring more advanced techniques to choose hyperparameters from cross validation results, taking into account variance, preferring more stable hyperparameter choices.
- implementing mini-batch SGD for the kernel algorithms.
- including the wine type (i.e. red or white) as a feature.

References

- [1] Paulo Cortez, António Cerdeira, Fernando Almeida, Telmo Matos, and José Reis. Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4):547–553, 2009. Smart Business Networks: Concepts and Empirical Evidence.
- [2] Shai Shalev-Shwartz, Yoram Singer, and Nathan Srebro. Pegasos: Primal estimated sub-gradient solver for svm. In *Proceedings of the 24th International Conference on Machine Learning*, ICML '07, page 807–814, New York, NY, USA, 2007. Association for Computing Machinery.
- [3] Hsuan-Tien Lin. A study on sigmoid kernels for svm and the training of non-psd kernels by smo-type methods. 2005.

Disclaimer: I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

The use of the first person plural is simply a stylistic choice, I am the sole author.